TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM

TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

KHOA CÔNG NGHỆ THÔNG TIN



NGUYỄN TRỌNG PHÚC – 52400028
LÊ KHẢI HOÀN – 52400114
TRẦN HỮU LONG - 52400138

LEARN ABOUT ANIMATION & CANVAS AND BUILD AN ILLUSTRATED APPLICATION

BÁO CÁO GIỮA KỲ

PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM

TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

KHOA CÔNG NGHỆ THÔNG TIN



NGUYỄN TRỌNG PHÚC – 52400028
LÊ KHẢI HOÀN – 52400114
TRẦN HỮU LONG - 52400138

LEARN ABOUT ANIMATION & CANVAS AND BUILD AN ILLUSTRATED APPLICATION

BÁO CÁO GIỮA KỲ

PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG

Người hướng dẫn

Trần Đại Nhân

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Trước hết, nhóm chúng em xin gửi lời cảm ơn chân thành đến giảng viên môn Phát triển Ứng dụng Di động đã tận tình hướng dẫn và truyền đạt những kiến thức cần thiết trong suốt quá trình học tập. Những kiến thức và định hướng từ giảng viên đã giúp nhóm có cơ sở để nghiên cứu và hoàn thành đề tài "Learn about Animation & Canvas and Build an Illustrated Application".

Thông qua quá trình thực hiện đề tài, nhóm đã có cơ hội tìm hiểu sâu hơn về Animation và Canvas trong Android, đồng thời vận dụng kiến thức vào việc xây dựng ứng dụng minh họa. Quá trình này không chỉ giúp các thành viên củng cố kiến thức chuyên môn mà còn nâng cao kỹ năng lập trình, làm việc nhóm và giải quyết các vấn đề phát sinh trong quá trình phát triển ứng dụng.

Mặc dù nhóm đã cố gắng hoàn thiện đề tài một cách tốt nhất, báo cáo và sản phẩm minh họa khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được những nhận xét và góp ý từ giảng viên để có thể tiếp tục hoàn thiện kiến thức và kỹ năng trong thời gian tới.

Nhóm chúng em xin chân thành cảm ơn!

TP. Hồ Chí Minh, ngày 28 tháng 10 năm 2026

Tác giả

(Ký tên và ghi rõ họ tên)

Nguyễn Trọng Phúc

Lê Khải Hoàn

Trần Hữu Long

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH

TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi và được sự hướng dẫn khoa học của Trần Đại Nhân. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Dự án còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung Dự án của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày 28 tháng 10 năm 2026

Tác giả

(Ký tên và ghi rõ họ tên)

Nguyễn Trọng Phúc

Lê Khải Hoàn

Trần Hữu Long

LEARN ABOUT ANIMATION & CANVAS AND BUILD AN ILLUSTRATED APPLICATION

TÓM TẮT

Đề tài "Learn about Animation & Canvas and Build an Illustrated Application" tập trung nghiên cứu hai kỹ thuật quan trọng trong phát triển giao diện Android là Animation và Canvas. Về Animation, nhóm tìm hiểu các hiệu ứng phổ biến như di chuyển, xoay, thay đổi kích thước, độ trong suốt và hiệu ứng nảy; đồng thời nghiên cứu các thành phần như ObjectAnimator, ValueAnimator, AnimatorSet và Interpolator. Về Canvas, đề tài tập trung vào Canvas, Paint, Path, Custom View, onDraw(), invalidate() và xử lý sự kiện cảm ứng, qua đó làm rõ cách Android hỗ trợ việc vẽ và cập nhật nội dung đồ họa.

Bên cạnh việc nghiên cứu lý thuyết, đề tài còn tìm hiểu cách kết hợp Animation với Canvas. Trong đó, Animation được sử dụng để cập nhật các thuộc tính của đối tượng theo thời gian, sau đó yêu cầu Custom View vẽ lại thông qua invalidate() và onDraw(). Cơ chế này cho phép tạo ra các đối tượng đồ họa có khả năng chuyển động và tương tác trực quan trên màn hình.

Trên cơ sở những kiến thức đã nghiên cứu, nhóm xây dựng một ứng dụng Android minh họa nhằm trực quan hóa các kỹ thuật Animation và Canvas. Thông qua quá trình nghiên cứu và triển khai sản phẩm, đề tài giúp củng cố kiến thức về xử lý đồ họa, hiệu ứng chuyển động và tương tác người dùng trong phát triển ứng dụng Android, đồng thời tạo nền tảng để áp dụng các kỹ thuật này vào những ứng dụng phức tạp hơn trong tương lai.

MỤC LỤC

DANH MỤC HÌNH VẼ vi

DANH MỤC CÁC CHỮ VIẾT TẮT viii

CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI 1

1.1 Lý do chọn đề tài 1

1.2 Mục tiêu thực hiện đề tài 1

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 2

2.1 Tổng quan về Animation trong Android 2

2.1.1 Khái niệm Animation 2

2.1.2 Các dạng Animation phổ biến 2

2.1.3 View Animation và Property Animation 3

2.1.4 Các lớp hỗ trợ Animation 3

2.1.5 Interpolator 7

2.1.6 Các phương thức điều khiển Animation 7

2.2 Tổng quan về Canvas trong Android 7

2.2.1 Khái niệm Canvas 7

2.2.2 Hệ tọa độ Canvas 8

2.2.3 Paint 9

2.2.4 Các phương thức vẽ của Canvas 9

2.2.5 Path 9

2.2.6 Custom View và phương thức onDraw() 10

2.2.7 Cơ chế cập nhật giao diện 11

2.2.8 Xử lý sự kiện cảm ứng 11

2.3 Kết hợp Animation và Canvas 12

2.3.1 Nguyên lý kết hợp 12

2.3.2 Sử dụng ValueAnimator với Canvas 12

2.3.3 Biến đổi hệ tọa độ Canvas 14

2.3.4 Một số lưu ý khi triển khai 14

2.4 Một số kỹ thuật mở rộng 15

CHƯƠNG 3. XÂY DỰNG ỨNG DỤNG MINH HỌA ANIMATION VÀ CANVAS 15

3.1 Giới thiệu tổng quan ứng dụng 15

3.2 Kiến trúc và tổ chức chức năng 16

3.3 Phân hệ Animation Studio 17

3.4 Phân hệ Canvas Drawing Lab 19

3.5 Phân hệ Particle Fireworks Lab 22

3.6 Đánh giá kết quả thực hiện 23

CHƯƠNG 4. KẾT LUẬN 24

CHƯƠNG 5. PHÂN CÔNG NHIỆM VỤ VÀ ĐÁNH GIÁ TỶ LỆ ĐÓNG GÓP 25

TÀI LIỆU THAM KHẢO 26

DANH MỤC HÌNH VẼ

Hình 2.1.4.1. Minh họa sử dụng ObjectAnimator và AnimatorSet để điều khiển chuyển động 4

Hình 2.1.4.2. Kết quả mô phỏng chuyển động của đối tượng bằng ObjectAnimator 4

Hình 2.1.4.3. Đoạn mã điều khiển tiến trình Replay bằng ValueAnimator 5

Hình 2.1.4.4. Kết quả mô phỏng tiến trình phát lại hoạt ảnh 5

Hình 2.1.4.5. Đoạn mã kết hợp hiệu ứng phóng to và xoay bằng AnimatorSet 6

Hình 2.1.4.6 Kết quả thực hiện đồng thời hiệu ứng Scale và Rotate 6

Hình 2.2.2.1. Đoạn mã xây dựng lưới tọa độ bằng Canvas 8

Hình 2.2. Minh họa hệ tọa độ hai chiều trong Android Canvas 8

Hình 2.2.5.1 Đoạn mã xây dựng hình tam giác bằng Path 10

Hình 2.2.5.2 Kết quả vẽ hình tam giác trên Canvas 10

Hình 2.2.6.1. Đoạn mã xây dựng Custom View và ghi đè phương thức onDraw() 11

Hình 2.2.6.2 Kết quả hiển thị đối tượng đồ họa bằng Custom View 11

Hình 2.2.8.1 Đoạn mã xử lý sự kiện cảm ứng thông qua onTouchEvent() 12

Hình 2.3.2.1 Đoạn mã kết hợp ValueAnimator với Canvas 13

Hình 2.3.2.2. Kết quả mô phỏng chuyển động của quả bóng trên Canvas 13

Hình 2.3.3.1. Đoạn mã thực hiện phép xoay đối tượng bằng Canvas 14

Hình 2.3.3.2 Kết quả thực hiện phép xoay và khôi phục trạng thái Canvas 14

Hình 3.1.1. Giao diện chính của ứng dụng Animation & Canvas Playground 16

Hình 3.2.1. Sơ đồ tổ chức các phân hệ của ứng dụng 17

Hình 3.3.1. Giao diện thực hiện hiệu ứng Combo Set trong Animation Studio 18

Hình 3.3.2. Đồ thị Interpolator trong Animation Studio 19

Hình 3.5.1. Giao diện vẽ và chỉnh sửa đối tượng bằng Canvas 20

Hình 3.6. Chức năng Replay Path Animation trong Canvas Drawing Lab 21

Hình 3.7 – Giao diện mô phỏng hiệu ứng pháo hoa bằng hệ thống hạt nên đặt trong mục 23

DANH MỤC BẢNG BIỂU

Bảng 2.1.2.1. Các dạng Animation phổ biến trong Android 2

Bảng 2.1.3.1. So sánh View Animation và Property Animation 3

Bảng 2.1.5.1 Các loại Interpolator và ứng dụng 7

Bảng 2.1.6.1 Các phương thức điều khiển Animation.....7

Bảng 2.2.3.1 Các thuộc tính và phương thức của Paint 9

Bảng 2.2.4.1 Các phương thức vẽ cơ bản của Canvas.....9

Bảng 2.2.5.1 Các phương thức xử lý Path 10

Bảng 2.2.8.1 Các sự kiện cảm ứng trong Android.....12

Bảng 2.4.1 Một số kỹ thuật mở rộng Animation và Canvas.....15

DANH MỤC CÁC CHỮ VIẾT TẮT

Chữ viết tắt	Tên đầy đủ	Ý nghĩa tiếng Việt
API	Application Programming Interface	Giao diện lập trình ứng dụng
UI	User Interface	Giao diện người dùng
UX	User Experience	Trải nghiệm người dùng
XML	Extensible Markup Language	Ngôn ngữ đánh dấu mở rộng
SDK	Software Development Kit	Bộ công cụ phát triển phần mềm
IDE	Integrated Development Environment	Môi trường phát triển tích hợp
FPS	Frames Per Second	Số khung hình trên giây
2D	Two-Dimensional	Đồ họa hai chiều
3D	Three-Dimensional	Đồ họa ba chiều
RGB	Red, Green, Blue	Hệ màu đỏ, xanh lá, xanh dương
ARGB	Alpha, Red, Green, Blue	Hệ màu có kênh độ trong suốt
ms	Millisecond	Mili giây

CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI

1.1 Lý do chọn đề tài

Trong quá trình phát triển ứng dụng Android, giao diện và trải nghiệm người dùng là những yếu tố quan trọng ảnh hưởng đến tính trực quan và khả năng tương tác của ứng dụng. Bên cạnh các thành phần giao diện cơ bản, Animation giúp tạo ra các hiệu ứng chuyển động sinh động, trong khi Canvas cho phép lập trình viên chủ động vẽ và xử lý các đối tượng đồ họa theo yêu cầu.

Animation và Canvas có thể được sử dụng riêng biệt hoặc kết hợp với nhau để tạo ra nhiều hiệu ứng trực quan và tương tác trên ứng dụng Android. Việc tìm hiểu các kỹ thuật này giúp nhóm hiểu rõ hơn về cách tạo chuyển động, xử lý đồ họa, Custom View và sự tương tác của người dùng trên màn hình.

Xuất phát từ những lý do trên, nhóm lựa chọn đề tài "Learn about Animation & Canvas and Build an Illustrated Application" nhằm nghiên cứu các kiến thức cơ bản về Animation và Canvas, đồng thời áp dụng những kiến thức đã tìm hiểu để xây dựng một ứng dụng minh họa. Qua đó, nhóm có thể củng cố kiến thức về phát triển ứng dụng Android và nâng cao khả năng vận dụng lý thuyết vào thực tế.

1.2 Mục tiêu thực hiện đề tài

Đề tài hướng đến việc nghiên cứu và vận dụng các kỹ thuật Animation và Canvas trong phát triển ứng dụng Android, qua đó giúp làm rõ vai trò của hiệu ứng chuyển động và xử lý đồ họa trong việc xây dựng giao diện trực quan và tăng khả năng tương tác với người dùng. Nội dung nghiên cứu tập trung vào các dạng Animation phổ biến như di chuyển, xoay, thay đổi kích thước, độ trong suốt và hiệu ứng nảy, đồng thời tìm hiểu cách sử dụng các thành phần như ObjectAnimator, ValueAnimator, AnimatorSet và Interpolator để xây dựng và kiểm soát các hiệu ứng chuyển động. Bên cạnh đó, đề tài nghiên cứu các thành phần cơ bản của Canvas gồm Canvas, Paint, Path và Custom View, cũng như cơ chế vẽ và cập nhật nội dung thông

qua onDraw(), invalidate() và xử lý tương tác bằng onTouchEvent(). Trên cơ sở các kiến thức đã nghiên cứu, nhóm tiến hành xây dựng một ứng dụng Android minh họa nhằm thể hiện cách áp dụng và kết hợp Animation với Canvas trong thực tế, từ đó củng cố kiến thức chuyên môn và nâng cao khả năng vận dụng lý thuyết vào quá trình phát triển ứng dụng di động.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về Animation trong Android

2.1.1 Khái niệm Animation

Về Animation là kỹ thuật tạo ra sự thay đổi liên tục của một đối tượng theo thời gian, giúp giao diện Android sinh động và phản hồi trực quan với thao tác người dùng. Animation có thể thay đổi vị trí, góc xoay, kích thước, độ trong suốt hoặc các thuộc tính khác.

Ví dụ: biểu tượng trái tim phóng to rồi thu nhỏ khi người dùng nhấn nút thích; menu trượt vào màn hình; quả bóng nảy lên rồi rơi xuống.

2.1.2 Các dạng Animation phổ biến

Dạng	**Chức năng**	**Ví dụ**
Move / Translate	Di chuyển đối tượng theo trục X hoặc Y.	Mây trôi ngang màn hình.
Rotate	Xoay đối tượng quanh một điểm hoặc trục.	Cánh quạt hoặc bánh xe quay.
Scale	Phóng to hoặc thu nhỏ đối tượng.	Nút bấm lớn lên khi nhấn.
Fade / Alpha	Thay đổi độ trong suốt để hiện hoặc ẩn đối tượng.	Ngôi sao xuất hiện dần.
Bounce	Tạo cảm giác nảy ở cuối chuyển động; thường dùng BounceInterpolator.	Quả bóng nảy trên mặt đất.

Bảng 2.1.2.1. Các dạng Animation phổ biến trong Android

2.1.3 View Animation và Property Animation

Tiêu chí	View Animation	Property Animation
Mục đích	Tạo hiệu ứng hiển thị cho View.	Thay đổi giá trị thuộc tính theo thời gian.
Lớp thường dùng	TranslateAnimation, RotateAnimation, ScaleAnimation, AlphaAnimation.	ObjectAnimator, ValueAnimator, AnimatorSet.
Phạm vi	Chủ yếu là biến đổi hình ảnh hiển thị.	Có thể hoạt ảnh hóa nhiều thuộc tính và giá trị tùy chỉnh.
Ví dụ	Hiệu ứng đơn giản cho View.	Di chuyển View hoặc cập nhật dữ liệu vẽ Canvas.

Bảng 2.1.3.1. So sánh View Animation và Property Animation

Lưu ý: View Animation có thể làm View trông như đã di chuyển nhưng không thay đổi vị trí bố cục thực tế. Property Animation thay đổi thuộc tính được chỉ định; ví dụ translationX không tự thay đổi layout_margin hay kích thước bố cục.

2.1.4 Các lớp hỗ trợ Animation

a. ObjectAnimator

ObjectAnimator tự động thay đổi một thuộc tính của đối tượng. Đối tượng cần có thuộc tính phù hợp và setter tương ứng.

```
    private void playMove() {
        float d = maxHorizontalDistance();

        ObjectAnimator move = ObjectAnimator.ofFloat(
                target, View.TRANSLATION_X,
                ...values: 0f, d, -d, 0f
        );

        ObjectAnimator moveShadow = ObjectAnimator.ofFloat(
                shadow, View.TRANSLATION_X,
                ...values: 0f, d, -d, 0f
        );

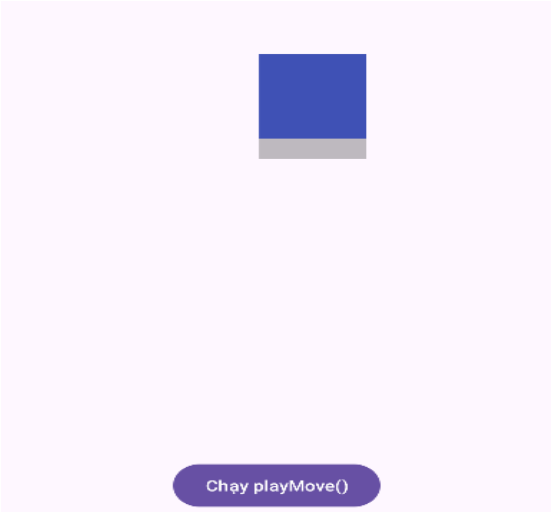
        applyRepeat(move);
        applyRepeat(moveShadow);

        AnimatorSet set = new AnimatorSet();
        set.playTogether(move, moveShadow);
        set.setDuration(duration * 2);
        set.setInterpolator(interpolator);

        start(set,  duration: duration * 2,
                info: "ObjectAnimator.ofFloat(view, \"translationX\", 0f, ...)");
    }
}
```

Hình 2.1.4.1. Minh họa sử dụng ObjectAnimator và AnimatorSet để điều khiển chuyển động

Đối tượng chính và bóng đổ cùng di chuyển sang phải, sang trái rồi trở về vị trí ban đầu. ObjectAnimator điều khiển thuộc tính TRANSLATION_X, còn AnimatorSet phối hợp hai hoạt ảnh chạy đồng thời.



Hình 2.1.4.2. Kết quả mô phỏng chuyển động của đối tượng bằng ObjectAnimator

b. ValueAnimator

ValueAnimator tính toán giá trị thay đổi theo thời gian nhưng không tự cập nhật thuộc tính của mọi View. Lập trình viên nhận giá trị mới qua addUpdateListener() rồi cập nhật dữ liệu và gọi invalidate() nếu cần vẽ lại Canvas.

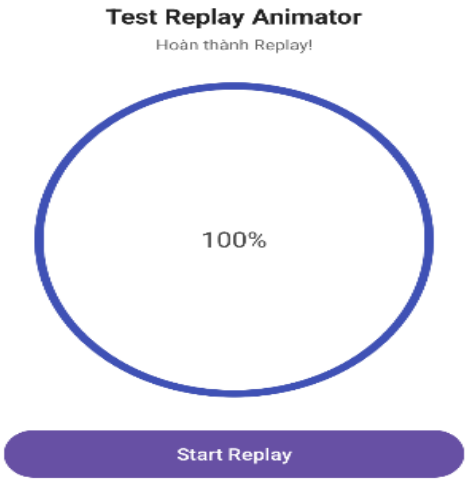
```
replayView = findViewById(R.id.replayView);
tvStatus = findViewById(R.id.tvStatus);
Button btnStart = findViewById(R.id.btnStart);

replayView.setTotalMs(3000L);
replayView.setOnComplete(() -> {
    if (tvStatus != null) {
        tvStatus.setText("Hoàn thành Replay!");
    }
});

btnStart.setOnClickListener( View v -> {
    if (tvStatus != null) {
        tvStatus.setText("Đang chạy Replay...");
    }
    replayView.startReplay();
});
```

Hình 2.1.4.3. Đoạn mã điều khiển tiến trình Replay bằng ValueAnimator

Test ValueAnimator cập nhật tiến độ phát lại nét vẽ từ 0f đến 1f. Mỗi lần tiến độ thay đổi, invalidate() yêu cầu DrawingView vẽ lại. Khi hoạt ảnh kết thúc, trạng thái phát lại được đặt về hoàn tất.



Hình 2.1.4.4. Kết quả mô phỏng tiến trình phát lại hoạt ảnh

c. AnimatorSet

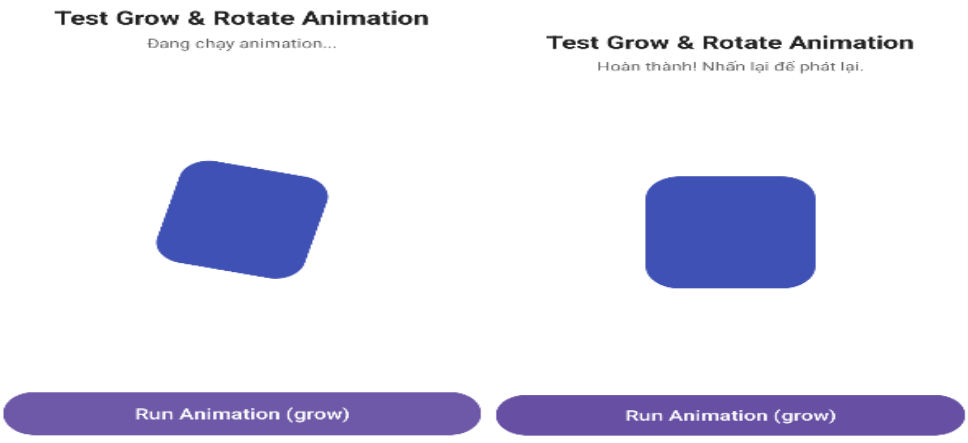
AnimatorSet cho phép phối hợp nhiều Animation. playTogether() chạy các hiệu ứng đồng thời; playSequentially() chạy lần lượt theo thứ tự.

```
target.setScaleX(1f);
target.setScaleY(1f);
target.setRotation(0f);

AnimatorSet grow = new AnimatorSet();

grow.playTogether(
        ObjectAnimator.ofFloat(target, View.SCALE_X, ...values: 1f, 1.4f),
        ObjectAnimator.ofFloat(target, View.SCALE_Y, ...values: 1f, 1.4f),
        ObjectAnimator.ofFloat(target, View.ROTATION, ...values: 0f, 180f)
);
```

Hình 2.1.4.5. Đoạn mã kết hợp hiệu ứng phóng to và xoay bằng AnimatorSet



Hình 2.1.4.6 Kết quả thực hiện đồng thời hiệu ứng Scale và Rotate

2.1.5 Interpolator

Interpolator quyết định tiến trình thay đổi của Animation theo thời gian. Các hiệu ứng có cùng thời lượng vẫn có thể tạo cảm giác khác nhau khi dùng Interpolator khác nhau.

Interpolator	**Đặc điểm**	**Ứng dụng**
LinearInterpolator	Thay đổi đều theo thời gian.	Chuyển động ổn định.
AccelerateInterpolator	Tăng tốc dần.	Đối tượng lấy tốc độ.
DecelerateInterpolator	Chậm dần khi đến cuối.	Đối tượng tiến đến điểm dừng.
AccelerateDecelerateInterpolator	Chậm ở đầu và cuối, nhanh hơn ở giữa.	Chuyển động giao diện tự nhiên.
BounceInterpolator	Nảy ở cuối chuyển động.	Quả bóng hoặc biểu tượng bật lên.
OvershootInterpolator	Vượt điểm đích rồi quay lại.	Hiệu ứng đàn hồi.

Bảng 2.1.5.1 Các loại Interpolator và ứng dụng

2.1.6 Các phương thức điều khiển Animation

Phương thức	**Ý nghĩa**
setDuration(long)	Thiết lập thời gian chạy, đơn vị mili giây.
setStartDelay(long)	Trì hoãn thời điểm bắt đầu.
setRepeatCount(int)	Thiết lập số lần lặp.
setRepeatMode(int)	Chọn cách lặp, ví dụ RESTART hoặc REVERSE.
start()	Bắt đầu Animation.
cancel()	Hủy Animation đang chạy.
end()	Đưa Animation đến trạng thái kết thúc.
addListener()	Theo dõi sự kiện bắt đầu, kết thúc hoặc hủy.
addUpdateListener()	Nhận giá trị cập nhật trong ValueAnimator.

Bảng 2.1.6.1 Các phương thức điều khiển Animation

2.2 Tổng quan về Canvas trong Android

2.2.1 Khái niệm Canvas

Canvas là bề mặt vẽ dùng để tạo hình ảnh bằng mã nguồn. Canvas thường được sử dụng bên trong Custom View để vẽ hình cơ bản, đường cong, văn bản và bitmap.

Ba thành phần cốt lõi: Canvas xác định nơi và thao tác vẽ; Paint xác định màu sắc, nét vẽ và kiểu chữ; Path mô tả hình dạng được tạo từ nhiều đoạn thẳng hoặc đường cong.

2.2.2 Hệ tọa độ Canvas

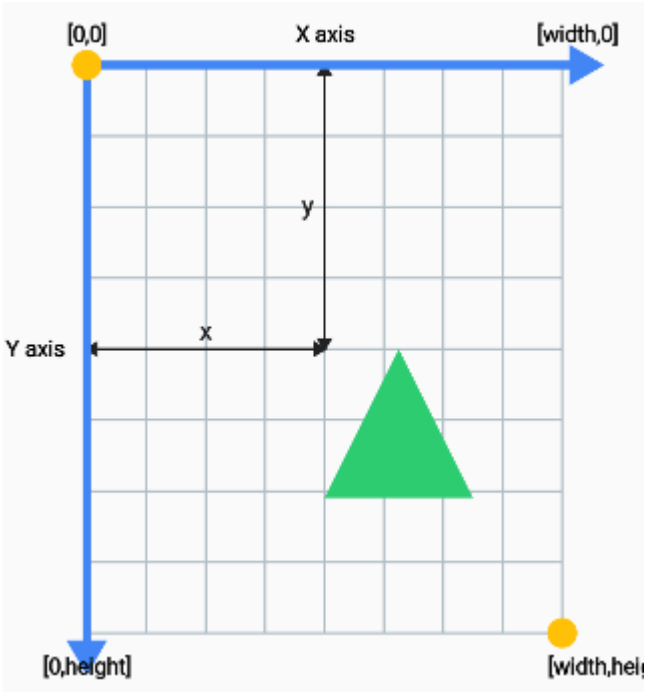
Gốc tọa độ (0, 0) thường nằm ở góc trên bên trái của View. Trục X tăng từ trái sang phải; trục Y tăng từ trên xuống dưới. Tọa độ và kích thước cần được tính theo kích thước thực tế của View để giao diện phù hợp với nhiều màn hình.

```
private void drawDotGrid(Canvas canvas, int w, int h) {
    float step = 20f * density;

    for (float x = step; x < w; x += step) {
        for (float y = step; y < h; y += step) {
            canvas.drawCircle(x, y,  radius: 1.3f * density, gridPaint);
        }
    }
}
```

Hình 2.2.2.1. Đoạn mã xây dựng lưới tọa độ bằng Canvas

Ví dụ vẽ hình tam giác màu xanh bằng đoạn thực tế từ DrawingVIew.java, phương thức drawDotGrid().



Hình 2.2. Minh họa hệ tọa độ hai chiều trong Android Canvas

2.2.3 Paint

Thuộc tính / phương thức	Công dụng
setColor()	Đặt màu vẽ.
setAlpha()	Đặt độ trong suốt.
setStyle(Paint.Style)	Chọn FILL, STROKE hoặc FILL_AND_STROKE.
setStrokeWidth()	Đặt độ dày nét.
setAntiAlias(true)	Làm mượt cạnh và đường vẽ.
setTextSize()	Đặt kích thước chữ.
setTypeface()	Chọn kiểu chữ.
setStrokeCap() / setStrokeJoin()	Thiết lập đầu nét và cách nối nét.

Bảng 2.2.3.1 Các thuộc tính và phương thức của Paint

2.2.4 Các phương thức vẽ của Canvas

Hàm	Chức năng
drawColor()	Tô màu nền.
drawLine() / drawPoint()	Vẽ đường thẳng hoặc điểm.
drawRect() / drawRoundRect()	Vẽ hình chữ nhật hoặc hình chữ nhật bo góc.
drawCircle() / drawOval()	Vẽ hình tròn hoặc hình elip.
drawArc()	Vẽ cung tròn hoặc phần hình quạt.
drawPath()	Vẽ hình dạng được mô tả bởi Path.
drawText()	Vẽ văn bản.
drawBitmap()	Vẽ hình ảnh bitmap.
translate() / rotate() / scale()	Biến đổi hệ tọa độ khi vẽ.
save() / restore()	Lưu và khôi phục trạng thái Canvas.
clipRect() / clipPath()	Giới hạn vùng được phép vẽ.

Bảng 2.2.4.1 Các phương thức vẽ cơ bản của Canvas

2.2.5 Path

Path mô tả các hình dạng phức tạp bằng cách kết hợp nhiều đoạn thẳng và đường cong. Sau khi xây dựng Path, dùng canvas.drawPath(path, paint) để vẽ.

Phương thức	Công dụng
moveTo(x, y)	Đặt điểm bắt đầu hoặc chuyển điểm hiện tại.
lineTo(x, y)	Vẽ đoạn thẳng đến điểm mới.
quadTo(...)	Tạo đường cong Bézier bậc hai.
cubicTo(...)	Tạo đường cong Bézier bậc ba.
close()	Nối điểm cuối với điểm đầu.
reset()	Xóa nội dung Path để sử dụng lại.

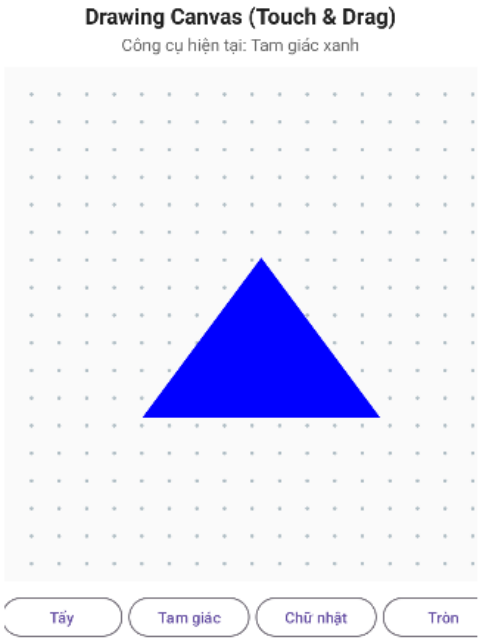
Bảng 2.2.5.1 Các phương thức xử lý Path

```
Path path = new Path();
path.moveTo( x: 20,  y: 100);
path.lineTo( x: 80,  y: 20);
path.lineTo( x: 140,  y: 100);
path.close();

Paint paint = new Paint(Paint.ANTI_ALIAS_FLAG);
paint.setColor(Color.GREEN);
paint.setStyle(Paint.Style.FILL);
canvas.drawPath(path, paint);
```

Hình 2.2.5.1 Đoạn mã xây dựng hình tam giác bằng Path

Ví dụ tạo một hình tam giác màu xanh bằng Path.



Hình 2.2.5.2 Kết quả vẽ hình tam giác trên Canvas

2.2.6 Custom View và phương thức onDraw()

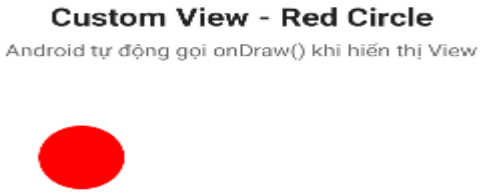
Custom View là lớp View do lập trình viên tự tạo bằng cách kế thừa android.view.View. Có thể ghi đè onDraw(Canvas canvas) để tự vẽ nội dung của View.

Nên khởi tạo Paint và Path từ trước, chẳng hạn trong constructor, rồi tái sử dụng trong onDraw() để tránh tạo nhiều đối tượng khi vẽ liên tục.

```
public static class MyDrawingView extends View {
    3 usages
    private final Paint paint = new Paint(Paint.ANTI_ALIAS_FLAG);
    public MyDrawingView(Context context) {
        super(context);
        init();
    }
    public MyDrawingView(Context context, @Nullable AttributeSet attrs) {
        super(context, attrs);
        init();
    }
    public MyDrawingView(Context context, @Nullable AttributeSet attrs, int defStyleAttr) {
        super(context, attrs, defStyleAttr);
        init();
    }
    3 usages
    private void init() {
        paint.setColor(Color.RED);
        paint.setStyle(Paint.Style.FILL);
    }
    @Override
    protected void onDraw(@NonNull Canvas canvas) {
        super.onDraw(canvas);
        canvas.drawColor(Color.WHITE);
        canvas.drawCircle( cx: 150,  cy: 150,  radius: 60, paint);
    }
}
```

Hình 2.2.6.1. Đoạn mã xây dựng Custom View và ghi đè phương thức onDraw()

Đoạn code tạo Custom View với nền trắng và hình tròn đỏ. Khi đưa View vào giao diện, Android gọi onDraw() khi cần hiển thị hoặc vẽ lại nội dung.



Hình 2.2.6.2 Kết quả hiển thị đối tượng đồ họa bằng Custom View

2.2.7 Cơ chế cập nhật giao diện

invalidate() thông báo rằng View cần được vẽ lại; Android sẽ gọi onDraw() vào thời điểm phù hợp. Không nên gọi trực tiếp onDraw() để cập nhật giao diện.

onSizeChanged() có thể được ghi đè để tính toán lại tọa độ hoặc tỷ lệ khi kích thước View thay đổi. Nếu cần yêu cầu vẽ lại từ luồng không phải giao diện, có thể dùng postInvalidate().

2.2.8 Xử lý sự kiện cảm ứng

onTouchEvent(MotionEvent event) xử lý các thao tác cảm ứng trên Custom View.

Sự kiện	Ý nghĩa
ACTION_DOWN	Ngón tay bắt đầu chạm.
ACTION_MOVE	Ngón tay di chuyển.
ACTION_UP	Ngón tay nhấc khỏi màn hình.
ACTION_CANCEL	Thao tác chạm bị hệ thống hủy.

Bảng 2.2.8.1 Các sự kiện cảm ứng trong Android

```
public boolean onTouchEvent(MotionEvent event) {
    if (event.getAction() == MotionEvent.ACTION_DOWN) {
        float x = event.getX();
        float y = event.getY();
        circleX = x;
        circleY = y;
        invalidate();
        return true;
    }
    return true;
}
```

Hình 2.2.8.1 Đoạn mã xử lý sự kiện cảm ứng thông qua onTouchEvent()

getX() và getY() lấy tọa độ chạm tương đối với View. Có thể dùng tọa độ này để đổi màu đối tượng, di chuyển hình vẽ hoặc kích hoạt Animation.

2.3 Kết hợp Animation và Canvas

2.3.1 Nguyên lý kết hợp

Canvas chịu trách nhiệm vẽ hình ảnh; Animation cập nhật các giá trị như vị trí, góc xoay, kích thước hoặc alpha theo thời gian. Custom View lưu các giá trị này rồi gọi invalidate() để yêu cầu vẽ lại. Trong onDraw(), Canvas vẽ hình theo trạng thái mới.

Chu trình cơ bản: Animation cập nhật giá trị → Custom View nhận giá trị → invalidate() → onDraw() vẽ lại → tiếp tục cập nhật cho đến khi Animation kết thúc.

2.3.2 Sử dụng ValueAnimator với Canvas

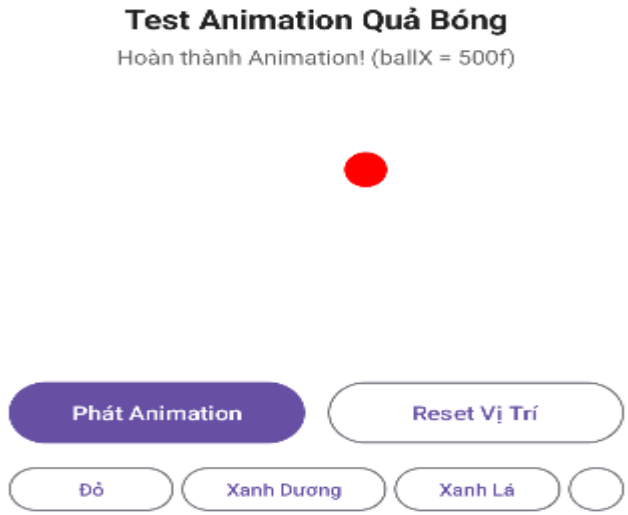
```
private final Paint ballPaint = new Paint(Paint.ANTI_ALIAS_FLAG);
public BallView(Context context) {
    super(context);
    ballPaint.setColor(Color.RED);
    ballPaint.setStyle(Paint.Style.FILL);

    post(this::startBallAnimation);
}
2 usages
private void startBallAnimation() {
    ValueAnimator animator = ValueAnimator.ofFloat( ...values: 50f, 500f);
    animator.setDuration(2000);
    animator.setInterpolator(new LinearInterpolator());

    animator.addUpdateListener( ValueAnimator animation -> {
        ballX = (float) animation.getAnimatedValue();
        invalidate();
    });
    animator.start();
}
```

Hình 2.3.2.1 Đoạn mã kết hợp ValueAnimator với Canvas

Đây là ví dụ minh họa cơ chế. Trong ứng dụng thực tế, cần khởi tạo màu cho ballPaint, giới hạn ballX theo chiều rộng View và hủy Animation khi không còn cần thiết.



Hình 2.3.2.2. Kết quả mô phỏng chuyển động của quả bóng trên Canvas

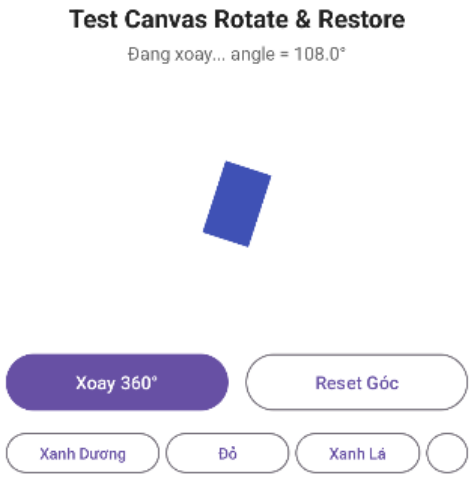
2.3.3 Biến đổi hệ tọa độ Canvas

```
int saveCount = canvas.save();
canvas.rotate(angle, centerX, centerY);
// Vẽ các bộ phận của đối tượng quanh tâm.
canvas.drawRect(left, top, right, bottom, paint);

canvas.restoreToCount(saveCount);
```

Hình 2.3.3.1. Đoạn mã thực hiện phép xoay đối tượng bằng Canvas

angle là góc xoay được cập nhật bởi Animation. save() lưu trạng thái Canvas; restoreToCount() khôi phục trạng thái trước khi xoay, tránh ảnh hưởng đến những hình vẽ khác.



Hình 2.3.3.2 Kết quả thực hiện phép xoay và khôi phục trạng thái Canvas

2.3.4 Một số lưu ý khi triển khai

- Không tạo mới nhiều Paint và Path trong mỗi lần onDraw(); nên tái sử dụng đối tượng.
- Dùng invalidate() để yêu cầu vẽ lại, không gọi trực tiếp onDraw().
- Hủy hoặc dừng Animation khi View không còn cần hoạt động.
- Tính toán tọa độ dựa trên chiều rộng và chiều cao thực tế của View.
- Dùng save() và restore() để quản lý các phép biến đổi Canvas.

2.4 Một số kỹ thuật mở rộng

Tính năng	API liên quan	Ví dụ
Đổi màu theo thời gian	ValueAnimator, ArgbEvaluator	Mặt trời đổi màu.
Di chuyển theo đường cong	Path, PathMeasure, ValueAnimator	Vật thể đi theo quỹ đạo.
Gradient	LinearGradient, Paint.setShader()	Bầu trời chuyển sắc.
Kéo thả đối tượng	onTouchEvent(), invalidate()	Di chuyển ngôi sao bằng ngón tay.
Giới hạn vùng vẽ	clipRect(), clipPath()	Tạo vùng hiển thị riêng.
Hiện ảnh dần	drawBitmap(), Paint.setAlpha()	Ảnh xuất hiện từ từ.
Điều khiển hoạt ảnh	start(), cancel(), reverse()	Chạy, dừng, đảo chiều hiệu ứng.

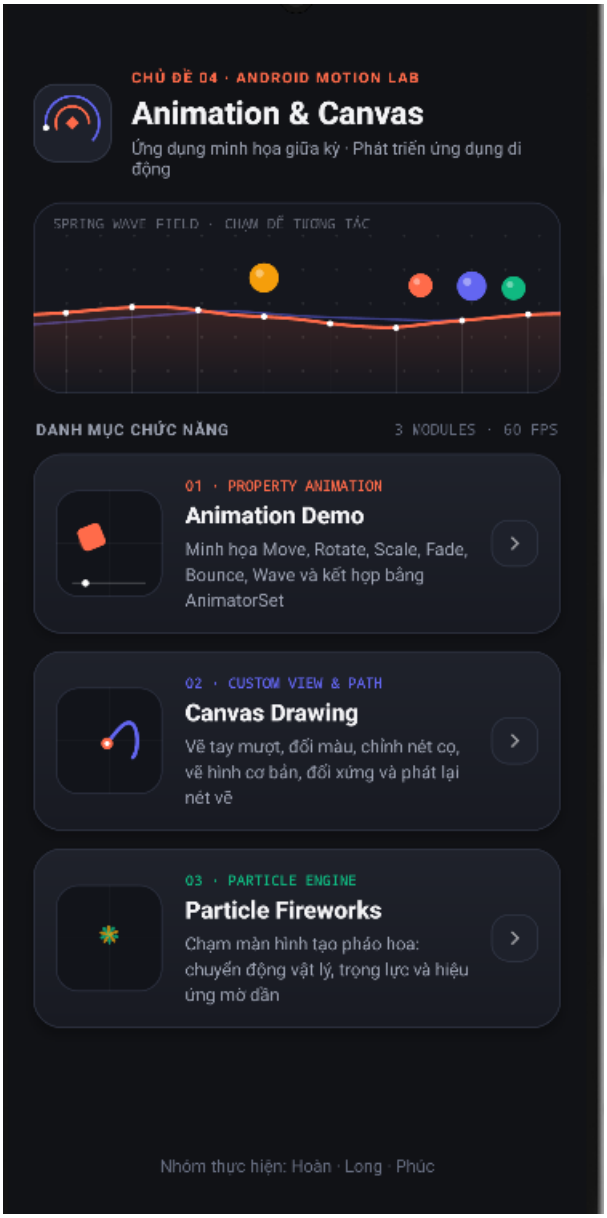
Bảng 2.4.1 Một số kỹ thuật mở rộng Animation và Canvas

CHƯƠNG 3. XÂY DỰNG ỨNG DỤNG MINH HỌA ANIMATION VÀ CANVAS

3.1 Giới thiệu tổng quan ứng dụng

Animation & Canvas Playground là ứng dụng được phát triển trên nền tảng Android nhằm minh họa các kỹ thuật Animation và Canvas thông qua những chức năng đồ họa có tính tương tác. Ứng dụng được xây dựng bằng ngôn ngữ Java, kết hợp các thành phần giao diện Android và Custom View để thực hiện các hiệu ứng chuyển động, vẽ hình và mô phỏng hệ thống hạt.

Hệ thống được tổ chức thành ba phân hệ chính gồm Animation Studio, Canvas Drawing Lab và Particle Fireworks Lab. Thông qua các phân hệ này, người dùng có thể trực tiếp thao tác, quan sát kết quả và tìm hiểu nguyên lý hoạt động của những kỹ thuật đồ họa trong Android.



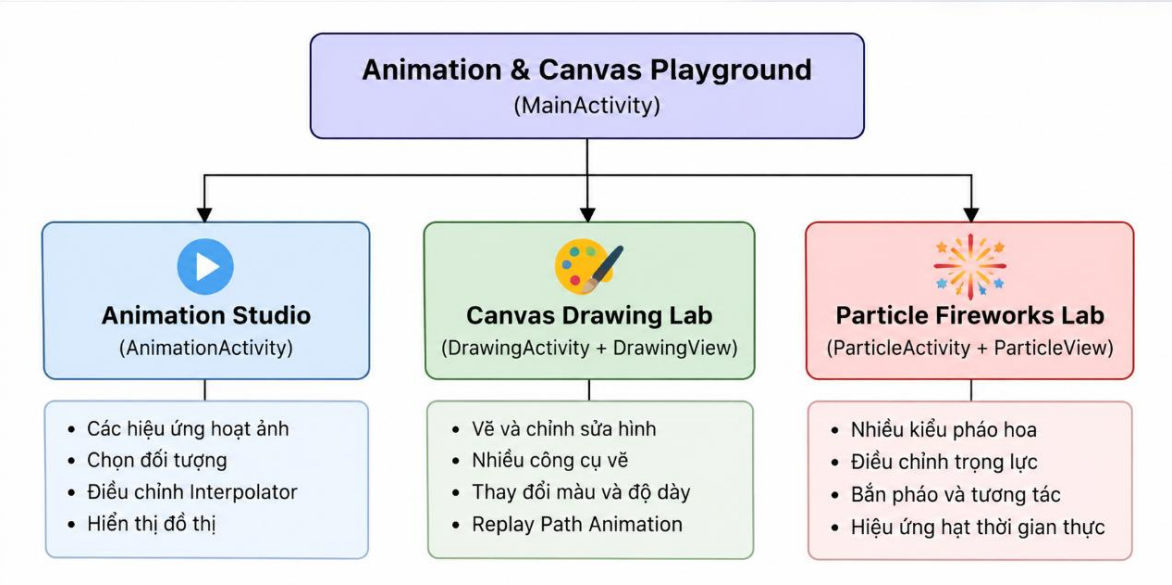
Hình 3.1.1. Giao diện chính của ứng dụng Animation & Canvas Playground

3.2 Kiến trúc và tổ chức chức năng

Ứng dụng được thiết kế theo cấu trúc gồm một màn hình chính và ba phân hệ chức năng độc lập. Màn hình chính đóng vai trò điều hướng, cho phép người dùng truy cập từng khu vực minh họa. Mỗi phân hệ được triển khai thông qua các lớp

Activity và View chuyên biệt, đảm nhiệm việc xử lý giao diện, tương tác và hiển thị đồ họa.

Cụ thể, AnimationActivity quản lý các hiệu ứng chuyển động; DrawingActivity kết hợp DrawingView để xử lý hoạt động vẽ; ParticleActivity phối hợp với ParticleView và lớp Particle để mô phỏng chuyển động của hệ thống hạt.

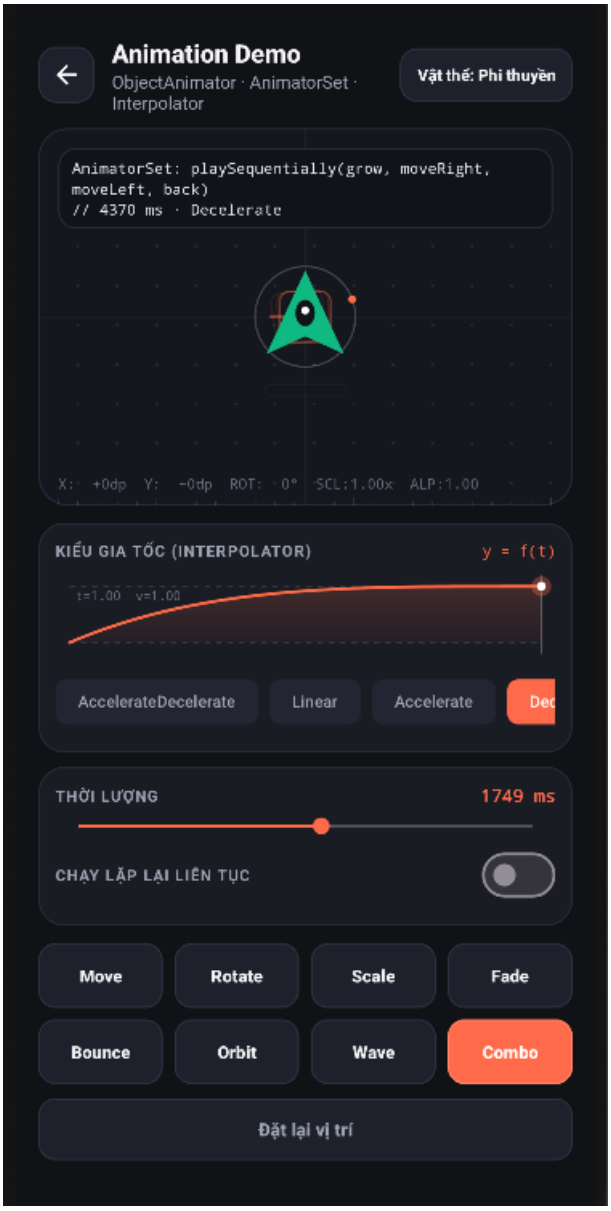


Hình 3.2.1. Sơ đồ tổ chức các phân hệ của ứng dụng

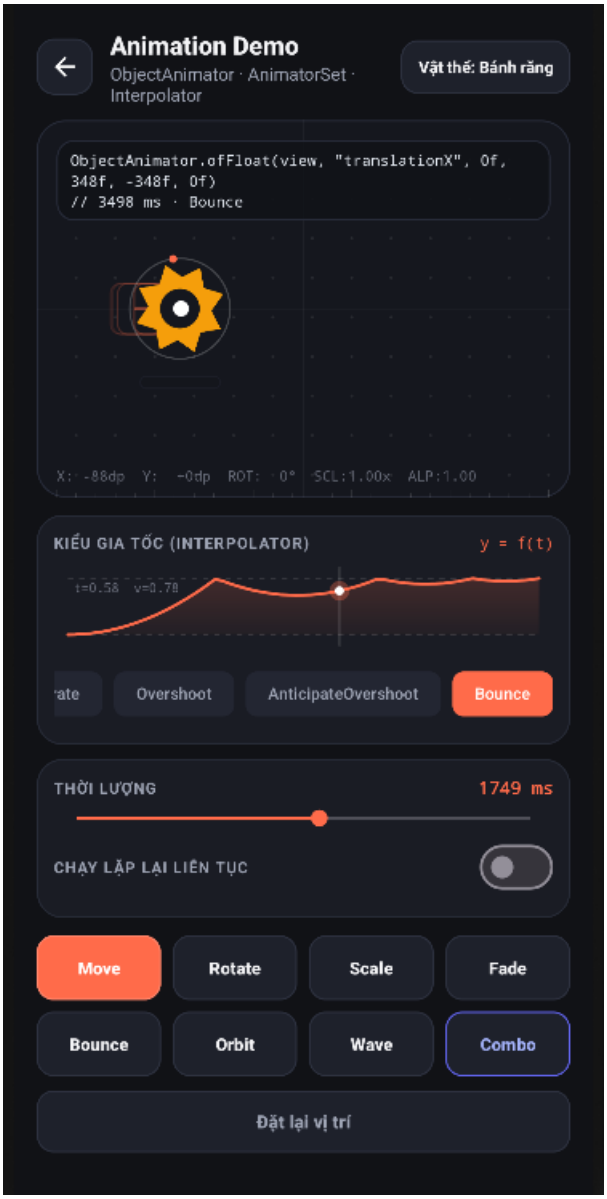
3.3 Phân hệ Animation Studio

Phân hệ Animation Studio được xây dựng nhằm minh họa các kỹ thuật Property Animation trong Android. Hệ thống hỗ trợ tám hiệu ứng gồm Move, Rotate 3D, Scale, Fade, Bounce, Orbit, Elastic Wave và Combo Set. Người dùng có thể thay đổi đối tượng, lựa chọn bộ nội suy, điều chỉnh thời lượng và chế độ lặp để quan sát sự khác biệt giữa các chuyển động.

Phân hệ sử dụng ObjectAnimator để thay đổi các thuộc tính của đối tượng, ValueAnimator để tính toán giá trị chuyển động và AnimatorSet để phối hợp nhiều hoạt ảnh. Ngoài ra, đồ thị Interpolator được xây dựng bằng Custom View giúp trực quan hóa quy luật biến thiên của chuyển động theo thời gian.



Hình 3.3.1. Giao diện thực hiện hiệu ứng Combo Set trong Animation Studio

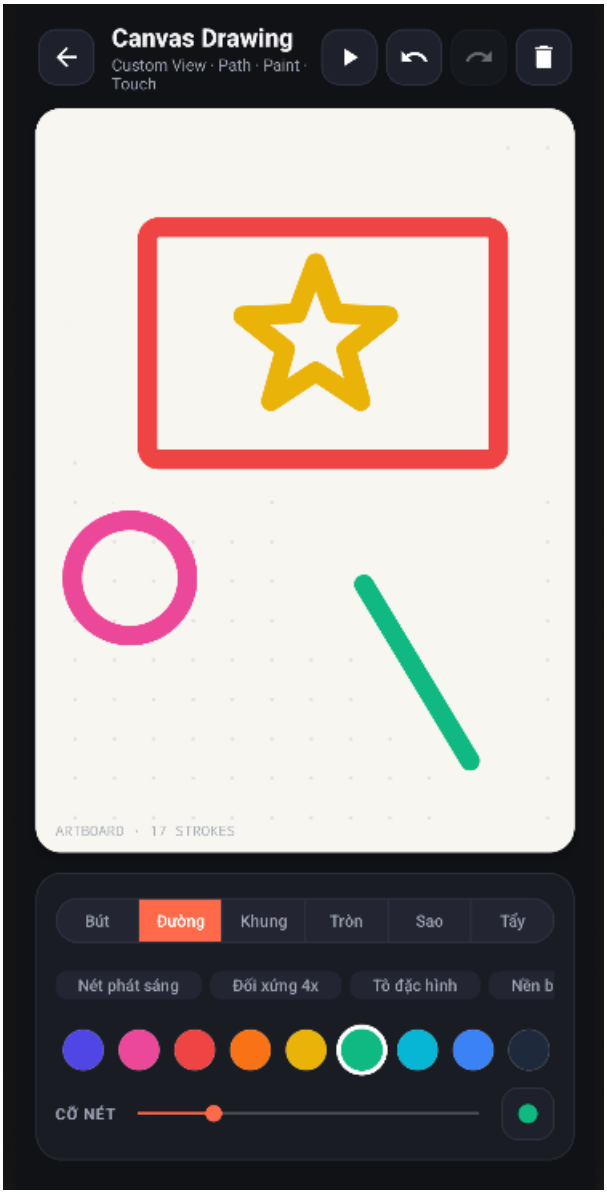


Hình 3.3.2. Đồ thị Interpolator trong Animation Studio

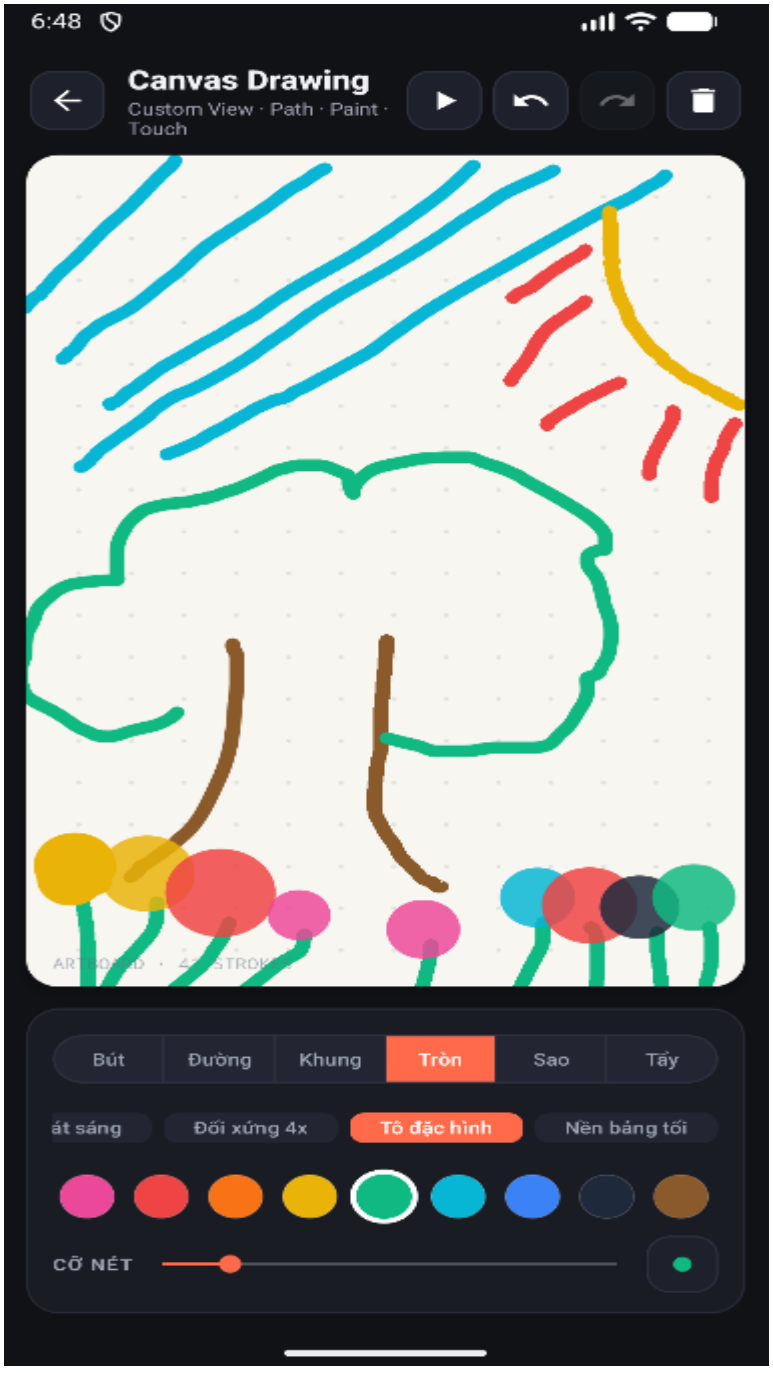
3.4 Phân hệ Canvas Drawing Lab

Phân hệ Canvas Drawing Lab cung cấp môi trường vẽ đồ họa trực tiếp trên màn hình cảm ứng. Người dùng có thể lựa chọn các công cụ vẽ tự do, đường thẳng, hình chữ nhật bo góc, hình tròn, ngôi sao và tẩy, đồng thời điều chỉnh màu sắc, độ dày và kiểu hiển thị nét vẽ.

Hệ thống sử dụng Canvas, Paint và Path để xây dựng các đối tượng đồ họa. Các thao tác cảm ứng được tiếp nhận thông qua onTouchEvent(), kết hợp invalidate() để cập nhật nội dung hiển thị. Chức năng Replay Path Animation sử dụng ValueAnimator và PathMeasure nhằm tái hiện quá trình hình thành nét vẽ, qua đó thể hiện sự kết hợp giữa hoạt ảnh và xử lý đồ họa Canvas.



Hình 3.5.1. Giao diện vẽ và chỉnh sửa đối tượng bằng Canvas

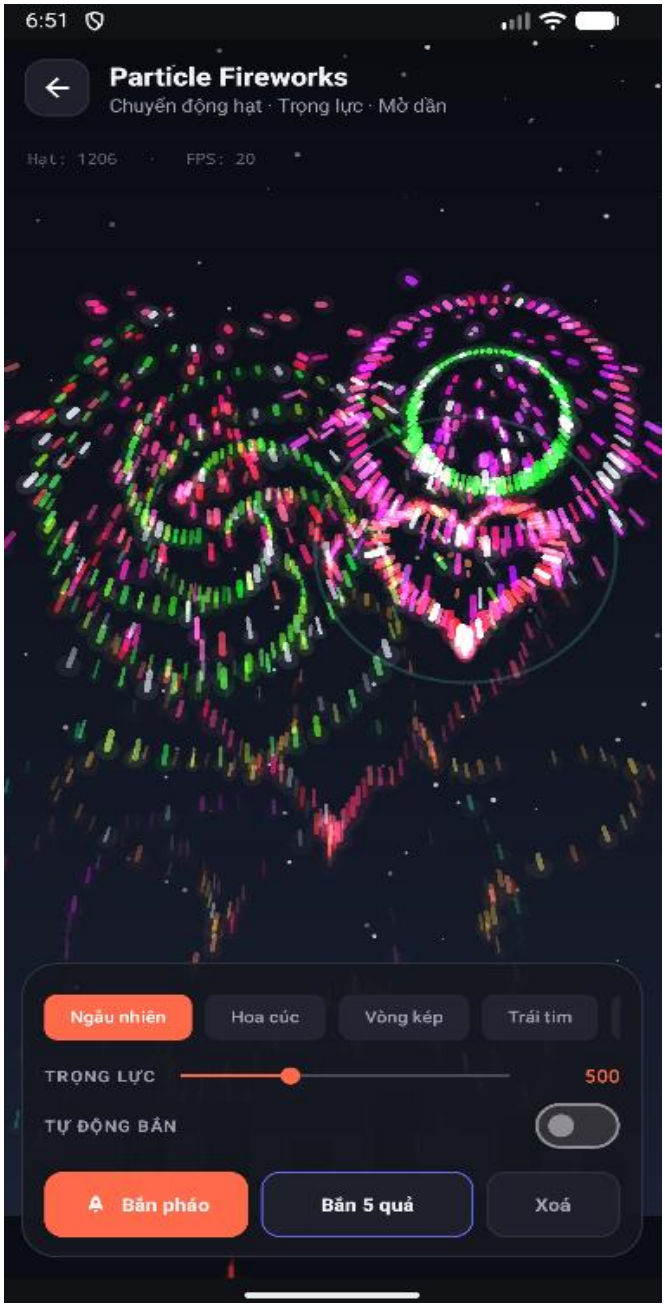


Hình 3.6. Chức năng Replay Path Animation trong Canvas Drawing Lab

3.5 Phân hệ Particle Fireworks Lab

Phân hệ Particle Fireworks Lab mô phỏng các hiệu ứng pháo hoa thông qua hệ thống hạt đồ họa. Ứng dụng hỗ trợ sáu kiểu hiệu ứng gồm Mix, Peony Sphere, Double Ring, Heart, Star và Spiral Galaxy. Người dùng có thể lựa chọn kiểu pháo hoa, điều chỉnh trọng lực, kích hoạt bắn liên tiếp hoặc tương tác trực tiếp trên vùng mô phỏng.

Hệ thống sử dụng lớp Particle để lưu trữ trạng thái của từng hạt, bao gồm vị trí, vận tốc và thời gian tồn tại. Lớp ParticleView thực hiện cập nhật trạng thái theo thời gian và sử dụng Canvas để hiển thị các hạt trên màn hình. Việc kết hợp các phép tính chuyển động, hiệu ứng ánh sáng và cơ chế vẽ lại liên tục tạo nên các hoạt ảnh pháo hoa có tính trực quan.



Hình 3.7 – Giao diện mô phỏng hiệu ứng pháo hoa bằng hệ thống hạt

3.6 Đánh giá kết quả thực hiện

Thông qua quá trình xây dựng ứng dụng Animation & Canvas Playground, nhóm đã triển khai được ba phân hệ minh họa các kỹ thuật Animation, Canvas và tương tác cảm ứng trên nền tảng Android. Các chức năng được tổ chức theo từng

nhóm kỹ thuật, giúp thể hiện rõ vai trò của Property Animation, Custom View, Canvas, Paint và Path trong quá trình phát triển giao diện đồ họa.

Ứng dụng không chỉ mô phỏng những hiệu ứng chuyển động cơ bản mà còn mở rộng sang các chức năng vẽ tương tác và mô phỏng hệ thống hạt. Qua đó, sản phẩm đáp ứng mục tiêu xây dựng ứng dụng minh họa, đồng thời tạo cơ sở thực hành và nghiên cứu các kỹ thuật đồ họa trong phát triển ứng dụng Android.

CHƯƠNG 4. KẾT LUẬN

Thông qua quá trình nghiên cứu và thực hiện đề tài "Learn about Animation & Canvas and Build an Illustrated Application", nhóm đã có cơ hội tìm hiểu các kiến thức nền tảng về Animation và Canvas trong phát triển ứng dụng Android. Đề tài đã giúp nhóm hiểu rõ hơn về các kỹ thuật tạo hiệu ứng chuyển động, phương pháp xử lý và vẽ đồ họa, cũng như cơ chế kết hợp Animation với Canvas để xây dựng các đối tượng có khả năng chuyển động và tương tác trên giao diện ứng dụng. Bên cạnh việc nghiên cứu lý thuyết, nhóm đã vận dụng những kiến thức thu được vào quá trình xây dựng ứng dụng minh họa. Thông qua các chức năng được triển khai, nhóm có thể quan sát và hiểu rõ hơn cách các thành phần Animation, Canvas, Paint, Path và Custom View phối hợp với nhau trong thực tế. Quá trình thực hiện cũng giúp các thành viên củng cố kỹ năng lập trình Android, khả năng giải quyết vấn đề và kinh nghiệm làm việc nhóm. Mặc dù đã đạt được những kết quả nhất định, đề tài vẫn còn một số hạn chế về phạm vi chức năng và khả năng mở rộng của ứng dụng. Trong tương lai, nhóm định hướng tiếp tục cải thiện hiệu suất xử lý đồ họa, bổ sung các hiệu ứng chuyển động đa dạng và phát triển thêm những tính năng tương tác nhằm nâng cao trải nghiệm người dùng. Nhìn chung, đề tài đã giúp nhóm đạt được mục tiêu nghiên cứu và bước đầu vận dụng kiến thức về Animation và Canvas vào phát triển ứng dụng Android. Đây cũng là nền tảng kiến thức và kinh nghiệm hữu ích để nhóm tiếp tục tìm hiểu, xây dựng các ứng dụng di động có giao diện trực quan và tính tương tác cao hơn.

CHƯƠNG 5. PHÂN CÔNG NHIỆM VỤ VÀ ĐÁNH GIÁ TỶ LỆ ĐÓNG GÓP

STT	MSSV	Họ và tên	Nhiệm vụ	Đóng góp	Hoàn thành
1	52400028	Nguyễn Trọng Phúc	Nghiên cứu Animation, xây dựng Animation Demo, Main Screen và điều hướng ứng dụng	30%	100%
2	52400114	Lê Khải Hoàn	Nghiên cứu Canvas, xây dựng Drawing Demo và xử lý tương tác	30%	100%
3	52400138	Trần Hữu Long	Nghiên cứu kết hợp Animation và Canvas, xây dựng Particle Demo	30%	100%
4	Cả nhóm		Tích hợp hệ thống, kiểm thử, hoàn thiện báo cáo, slide và video demo	10%	100%
Tổng cộng				100%	100%

TÀI LIỆU THAM KHẢO

Tiếng Việt

[1] Trung tâm Tin học, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM, "[Học Lập trình Android] Bài 24: View Animation," 2016. [Online]. Available: https://csc.edu.vn/tin-tuc/Tin-Hoat-Dong/hoc-lap-trinh-android-bai-24-view-animation-394. [Accessed: Oct. 23, 2026].

[2] Trung tâm Đào tạo và Phát triển nguồn nhân lực, Trường Đại học Bách khoa, ĐHQG-HCM, "Lập trình Android cơ bản," 2022. [Online]. Available: https://cce.hcmut.edu.vn/lap-trinh-android. [Accessed: Oct. 23, 2026].

Tiếng Anh

[1] Android Developers, "Property Animation Overview," *Android Developers*. [Online]. Available: https://developer.android.com/develop/ui/views/animations/prop-animation. [Accessed: Oct. 23, 2026].

[2] Android Developers, "View Animation," *Android Developers*. [Online]. Available: https://developer.android.com/develop/ui/views/animations/view-animation. [Accessed: Oct. 23, 2026].

[3] Android Developers, "Create a Custom Drawing," *Android Developers*. [Online]. Available: https://developer.android.com/develop/ui/views/layout/custom-views/custom-drawing. [Accessed: Oct. 23, 2026].

[4] Android Developers, "Making a Custom View Interactive," *Android Developers*. [Online]. Available: https://developer.android.com/develop/ui/views/layout/custom-views/making-interactive. [Accessed: Oct. 23, 2026].